Chapter 20 - The WAV Sample Player

The WAV player streams a PCM `.wav` file straight from memory into the mixer. It is the highest-level engine on this machine: the program supplies a pointer to a complete WAV file (header plus sample data), the chip parses the header to discover the sample rate, bit depth, and channel count, and then plays the file. Unlike the Paula DMA engine (Chapter 21), the program does not have to choose a period or stage buffers - the player handles every detail of pacing and resampling.

This is the engine for short sound effects, digitised speech, and any longer audio asset that ships as a plain WAV file.

20.1 What the WAV player can show

Item	Value
Format	Standard RIFF/WAVE PCM
Bit depth	8 or 16 bits per sample, signed or unsigned
Channels	Mono or stereo
Sample rates	Any rate declared in the WAV header
Output	Routed to two adjacent SoundChip DAC channels
Pause/resume	Supported
Looping	Supported
Volume	Independent per-channel, `0`–`255`

20.2 The register block

The player lives at `0xF0BD8`–`0xF0BF3`.

Address	Name	Purpose
`0xF0BD8`	`WAV_PLAY_PTR`	Low 32 bits of the WAV data pointer.
`0xF0BDC`	`WAV_PLAY_LEN`	Length of the WAV data in bytes.
`0xF0BE0`	`WAV_PLAY_CTRL`	Control bits (see below).
`0xF0BE4`	`WAV_PLAY_STATUS`	Status bits (read-only).
`0xF0BE8`	`WAV_POSITION`	Current source frame position (read-only).
`0xF0BEC`	`WAV_PLAY_PTR_HI`	High 32 bits of the data pointer (for 64-bit addressing).
`0xF0BF0`	`WAV_CHANNEL_BASE`	Index of the left SoundChip DAC channel; the right channel uses `base + 1`.
`0xF0BF1`	`WAV_VOLUME_L`	Left volume, `0`–`255`.
`0xF0BF2`	`WAV_VOLUME_R`	Right volume, `0`–`255`.
`0xF0BF3`	`WAV_FLAGS`	Bit `0` = force mono (downmix to one channel).

20.2.1 `WAV_PLAY_CTRL` bits

Bit	Field	Meaning
0	`START`	Begin playback.
1	`STOP`	Stop playback.
2	`LOOP`	When playback reaches the end, restart from the beginning.
3	`PAUSE`	Pause without resetting the position.
4	`LOOP_APPLY_ONLY`	Update the loop bit on the current playback without re-triggering.

20.2.2 `WAV_PLAY_STATUS` bits

Bit	Field	Meaning
0	`BUSY`	Playback in progress.
1	`ERROR`	Last operation failed (bad header, range out of memory, etc.).
2	`PAUSED`	Currently paused.
3	`STEREO`	Active file is stereo.

20.3 Programming model

To play a WAV file:

1. Load the file (header + data) into memory.
2. Write the data address to `WAV_PLAY_PTR` (and `WAV_PLAY_PTR_HI` if the file lives above the 4 GB line).
3. Write the size in bytes to `WAV_PLAY_LEN`.
4. (Optional) configure `WAV_CHANNEL_BASE`, `WAV_VOLUME_L`, `WAV_VOLUME_R`, and `WAV_FLAGS`.
5. Write `1` to `WAV_PLAY_CTRL` to start.

The chip parses the WAV header at the supplied address, figures out the format, and routes the resampled stream into the two SoundChip DAC channels at `WAV_CHANNEL_BASE` and `WAV_CHANNEL_BASE + 1` (mono files use only the left channel).

To stop playback: write `2` to `WAV_PLAY_CTRL`. To pause: write `8`. To resume: write `1` again. To enable looping at the start: write `5` (`START | LOOP`).

20.4 BASIC keywords

There is no dedicated `WAV` keyword. To play a `.wav` from disk in one step, use `SOUND PLAY` (Chapter 22). For direct programming from BASIC, use `POKE`:

```
10 POKE &H000F0800, 1               : REM AUDIO_CTRL = on
20 BLOAD "beep.wav", &H200000
30 POKE &H000F0BD8, &H00200000       : REM WAV_PLAY_PTR
40 POKE &H000F0BDC, 12000            : REM WAV_PLAY_LEN
50 POKE &H000F0BF1, 200              : REM left volume
60 POKE &H000F0BF2, 200              : REM right volume
70 POKE &H000F0BE0, 1                : REM start
```

To stop the sound: `POKE &H000F0BE0, 2`.

20.5 Putting it together

The WAV player is the simplest way to play any audio asset that already exists as a file. It does the WAV header parsing, the format conversion, and the resampling itself; the program only has to point at the bytes. For repeatable sound effects in a game loop, point `WAV_PLAY_PTR` at a preloaded buffer and trigger it each time. For background music that already exists as a WAV recording, point at the buffer once and let the player loop.

The next chapter covers the Paula DMA engine, used when the program needs lower-level control over sample timing and DMA.